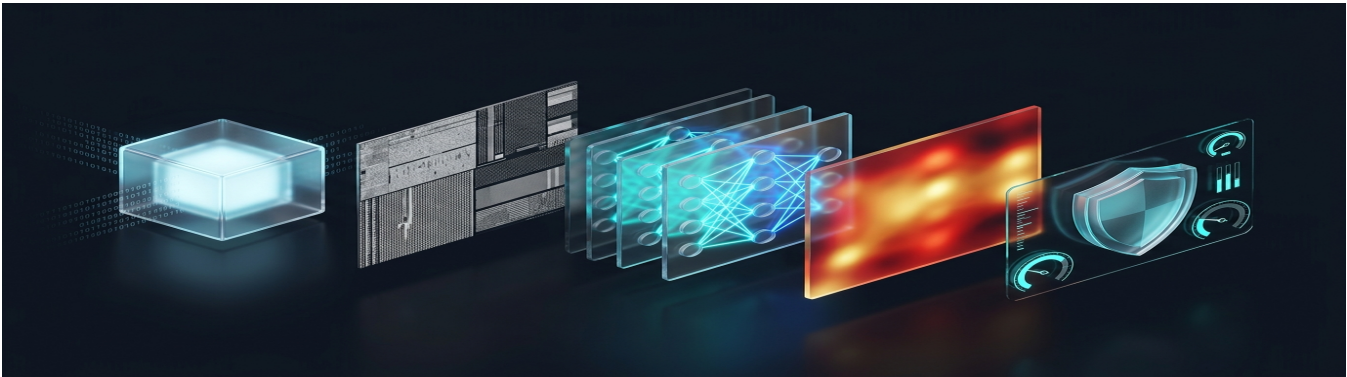


MALVISION: IMAGE-BASED MALWARE CLASSIFICATION

System Architecture, Deep Learning Methodology & Threat Hunting Platform

Architecture: ResNet-18 (Grayscale)	Accuracy: 97.86% (1,401 test samples)	Classes: 26 Malware Families
Dataset: Maling Benchmark (9,339 images)	Explainability: Layer4 Grad-CAM Heatmaps	Platform: Streamlit Web Dashboard



1. Executive Summary: The Big Idea in Simple Words

Instead of running a dangerous virus on a computer to see what harm it causes, **MalVision takes an 'X-ray photo' of the executable file and uses computer vision AI to spot the malware by its unique visual texture.**

The Airport Luggage Scanner Analogy: Airport security guards do not open every single bag by hand, nor do they wait for an item to detonate to know if it is dangerous. Instead, luggage passes through an X-ray scanner that converts contents into a density image where specialists spot prohibited shapes. **MalVision applies this exact principle to cybersecurity**—converting raw software binaries into visual textures where a Convolutional Neural Network (CNN) identifies the distinctive spatial fingerprints of malware families.

2. Why Traditional Antivirus Fails vs. Our Solution

Traditional antivirus mechanisms struggle against modern evasion techniques. **Signature matching** fails because polymorphic packers modify byte hashes daily. **Dynamic sandboxing** is slow (minutes per file), resource-heavy, and easily defeated by malware sleep/VM-detection checks.

Feature / Metric	Signature-Based AV	Dynamic Sandboxing	MalVision (Our System)
Analysis Latency	Milliseconds	3 to 10 Minutes per file	< 50 Milliseconds (Instant)
Execution Risk	Zero (passive hash lookup)	High (runs live hostile code)	100% Safe (zero code execution)
Evasion Resistance	None (1-byte edit defeats hash)	Vulnerable to sleep/anti-VM evasion	Immune (code is never run)
Polymorphic Resilience	Fails completely	Moderate	High (shared macro textures)
Explainability	None (binary yes/no rule)	Log telemetry only	Grad-CAM Visual Heatmaps

3. Core Methodology: Binary-to-Image Conversion

Every executable is an array of 8-bit unsigned bytes: $b_i \in [0, 255]$. In 8-bit grayscale imaging, pixel brightness is also defined as $p_i \in [0, 255]$ (0 = black, 255 = white). We map bytes directly to pixel intensities and reshape them into a 2D matrix using the **Nataraj et al. (2011)** heuristic. Width W is assigned based on file size, height is $H = \lceil N / W \rceil$, and trailing cells are zero-padded:

File Size	Width (W)	Rationale & Visual Behavior	File Size	Width (W)	Rationale & Visual Behavior
< 10 KB	32 px	Prevents 1-pixel line collapse.	100 - 200 KB	384 px	Accommodates multi-section droppers.
10 - 30 KB	64 px	Preserves row counts for dialers.	200 - 500 KB	512 px	Balanced aspect ratio for trojans.
30 - 60 KB	128 px	Captures header-to-code shifts.	500 KB - 1 MB	768 px	Preserves distinct code vs icon blobs.
60 - 100 KB	256 px	Standard resolution for PE tools.	> 1 MB	1024 px	High definition for complex suites.

Visual Textures of Executable Binary Sections

- **PE Header / DOS Stub:** High-contrast geometric alternating stripes with signature 'MZ' bytes and section offsets.
- **.text Section (Executable CPU Opcodes):** Fine-grained, peppered 'sand/static' pattern caused by frequent x86 jumps and calls.
- **.rdata & String Tables:** Distinct horizontal striated bands containing ASCII messages, URL paths, and registry keys.
- **.data Section:** Uniform dark blocks representing uninitialized variables, global state buffers, and zero-padding.
- **Encrypted / Packed Payloads:** Chaotic, high-entropy 'TV static snow' caused by encryption randomizing byte frequencies.

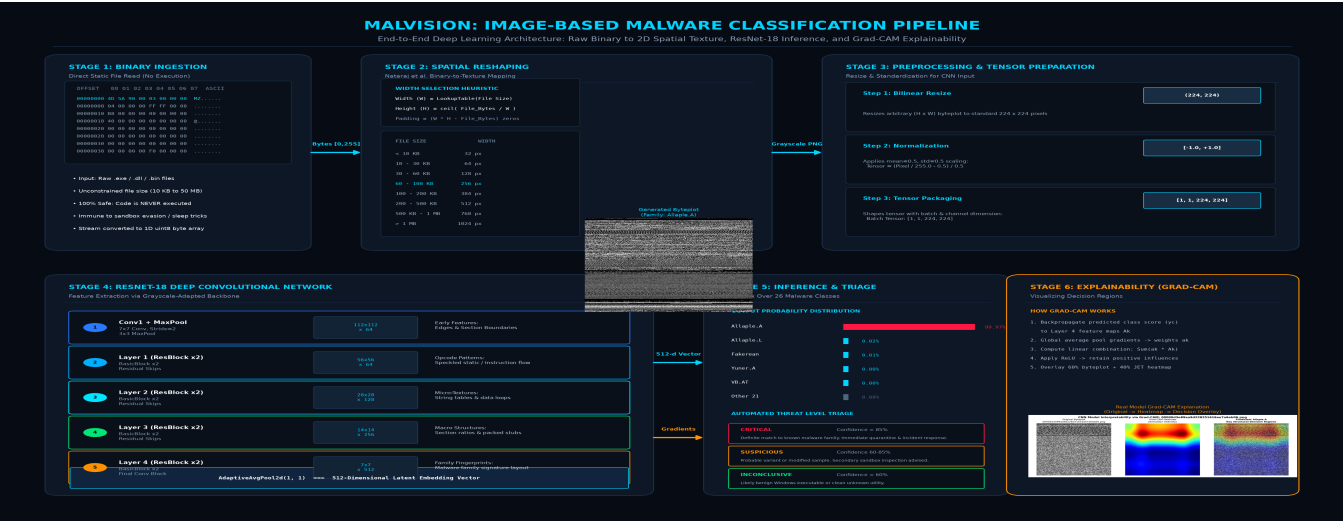
4. Deep Learning Backbone: ResNet-18 Architecture

We utilize an adapted **ResNet-18 (Residual Network)** with skip connections that prevent vanishing gradients. Pretrained ImageNet weights (originally 3 RGB channels of shape [64, 3, 7, 7]) were averaged across channels into shape [64, 1, 7, 7] to accept 1-channel grayscale input while preserving learned edge-detection filters.

Layer Block	Output Dimension	Spatial Semantics (What the Neural Network 'Sees')
Input Tensor	1 x 224 x 224	Normalized grayscale byteplot (mean=0.5, std=0.5 scaling).
Conv1 + MaxPool	64 x 112 x 112	Low-level primitives: Sharp borders between file sections and zero-padding runs.
Layer 1 (2x BasicBlock)	64 x 56 x 56	Opcode static: Instruction sequences and repetitive loop structures.
Layer 2 (2x BasicBlock)	128 x 28 x 28	Micro-textures: String tables, data boundaries, and import directory tables.
Layer 3 (2x BasicBlock)	256 x 14 x 14	Macro structure: Proportions of loader stub vs. encrypted payload sections.
Layer 4 (2x BasicBlock)	512 x 7 x 7	Malware family signatures: Distinct family architectural blueprint.
Adaptive AvgPool	512 x 1 x 1	Global pooling: Flattens 2D spatial features into a 512-dimensional vector.
Linear Classifier	26 Classes	Softmax probability distribution: $\sum_{j=1}^{26} P(\text{Family}_j) = 1.0$.

5. End-to-End System Architecture Blueprint

Below is the complete engineering pipeline tracing an uninspected binary from raw byte ingestion to final multi-class probability scoring and Grad-CAM interpretability:



6. Empirical Benchmark Results (Maling Dataset)

The model was evaluated using a stratified 70/15/15 split on the benchmark Maling dataset (9,339 samples across 26 families):

Evaluation Metric	Measured Value	Operational Significance & Reliability
Overall Test Accuracy	97.86%	1,371 correct classifications out of 1,401 unseen test samples.
Best Validation Loss	0.0574	Achieved at Epoch 3 checkpoint with early stopping safeguard.
Perfect F1-Score Families	18 of 26	100% precision & recall on AllApple, Fakerean, Yuner, VB.AT, etc.
Inference Latency	~35 ms	Sub-second classification on standard CPU hardware.

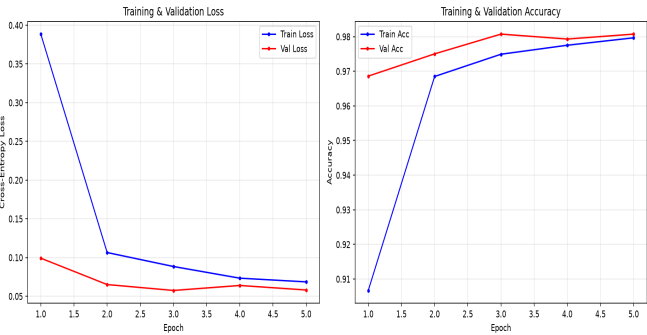


Figure A: Training & Validation Loss/Accuracy Curves

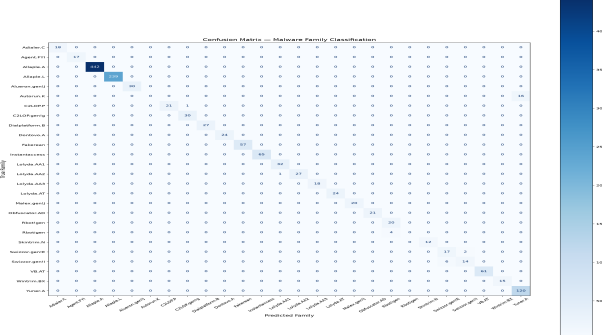


Figure B: 26-Class Confusion Matrix (Diagonal = True Positives)

7. Explainable AI: Grad-CAM Decision Heatmaps

To eliminate the 'black box' problem, MalVision implements **Grad-CAM (Gradient-Weighted Class Activation Mapping)**. It backpropagates gradients from the predicted class score to layer4 feature maps, calculates channel weights α_k , and projects a thermal overlay (Red = highest attention, Blue = lowest attention) over the original binary byteplot:

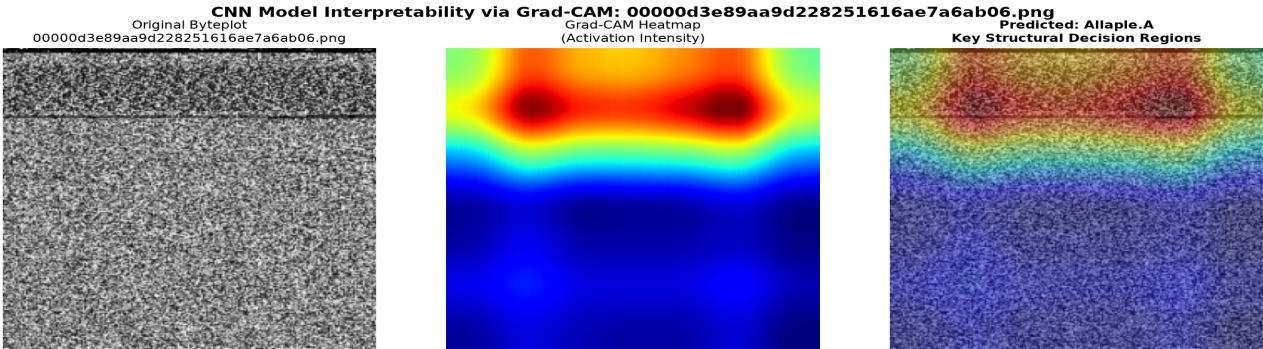


Figure C: 3-Panel Grad-CAM Output (Original Byteplot → Activation Heatmap → Decision Overlay)

8. Automated Threat Triage Levels

Threat Level	Confidence	Operational Verdict & Incident Response Action
CRITICAL	> 85%	Definite match to known malware family. Isolate host and quarantine binary immediately.
SUSPICIOUS	60% - 85%	Probable variant or modified sample. Route to secondary sandbox for behavioral audit.
INCONCLUSIVE	< 60%	Likely benign Windows executable (e.g. cmd.exe) or unindexed clean utility.

9. Project Defense & Viva Q&A; Cheat Sheet

Q: Why convert binaries to images instead of using disassembled opcodes (IDA Pro)?

A: Disassembling code requires complex static parsing, takes seconds to minutes, and fails when binaries are packed or obfuscated with anti-disassembly tricks. Image conversion takes under 10 milliseconds, reads raw byte values directly, and bypasses disassembly failures entirely.

Q: What prevents malware authors from modifying their code to evade your CNN?

A: While authors easily change variable names or insert junk opcodes to alter hash signatures, doing so only alters minor micro-textures. The macro-level section blueprint (e.g. tiny code stub + massive encrypted overlay) remains visually consistent across the entire family.

Q: How does the system handle clean / benign Windows files like cmd.exe?

A: Through confidence thresholding and entropy analysis. When clean software is tested, its visual layout does not match any malicious family. Softmax probabilities scatter thinly across all classes, keeping the top score below 60% and triggering an INCONCLUSIVE / BENIGN triage verdict.

Q: How did you adapt a pretrained 3-channel RGB model to 1-channel Grayscale?

A: We modified the first convolutional layer (conv1). The original ImageNet weights had shape [64, 3, 7, 7]. We averaged the weights across the 3 RGB channel dimensions into shape [64, 1, 7, 7], retaining all pretrained edge-detection filters while accepting single-channel grayscale input.